```
aravind@aravind-MP-Pavilion-g6-Notebook-PC ~ $ lein repl
nREPL server started on port 35534 on host 127.0.0.1
REPL-y 0.3.0
Clojure 1.5.1
    Docs: (doc function-name-here)
          (find-doc "part-of-name-here")
  Source: (source function-name-here)
 Javadoc: (javadoc java-object-or-class-here)
    Exit: Control+D or (exit) or (quit)
 Results: Stored in vars *1, *2, *3, an exception in *e

user=>
```

Figure 1: What a REPL for Clojure looks like

1. Hello world

Type the following command in a terminal:

```
$ lein repl
```

You'll get something similar to what is shown in Figure 1.

Now type the following command and press *Enter*. Notice how it's similar to the *System.out.println* familiar to all in Java.

```
user=> (println "Hello World!")
```

You'll get the output as:

```
user=> (println "Hello World!")
Hello World!
nil
```

2. Defining two variables and finding their sum

The following code defines two variables and stores their sum in yet another variable:

```
user=> (def a 432)
#'user/a
user=> (def b 123)
#'user/b
user=> (def c (+ a b))
#'user/c
user=> c
555
user=>
```

What are namespaces?

Namespaces are usually used to group symbols and identifiers around a particular functionality.

Clojure has a large amount of namespaces and they all come with many functions, thus making Clojure easy to use.

clojure.github.io/clojure gives the details of the functions and vars present in each namespace in Clojure, as well as their documentation.

We can change the namespace we want to use by giving the 'require' command in REPL. After using the 'require' command, just use the namespace name while referring to any function/variable described inside it.

The most recommended way to set up a new namespace at the top of a Clojure source file is to use the *ns* macro. By default, this will create a new namespace that will contain the mappings for classnames in *java.lang* and the functions in *clojure.core*.

Data structures in Clojure

Maps, vectors, sets and lists are the basic data structures that Clojure provides; the other data structures are present in Clojure collection hashes, StructMaps, ArrayMaps, etc.

In Clojure, there are seven primary abstractions whose data structures are as follows.

1. **Collection:** All data structures in Clojure follow the common collection abstraction. A collection is a value that you can use with the set of core collection functions.

   * *conj* is used to add an item to a collection:

```
user=> (conj '(237 4 2) 5)
(5 237 4 2)
```

   * *seq* is to get a sequence of a collection:

```
user=> (seq '(1 2 3) )
(1 2 3)
```

   * *count* helps in getting the number of items in a collection:

```
user=> (def a (seq '(1 2 3 4 5 6))
#'user/a
user=>(count a)
6
```

   * *empty* is used to obtain an empty instance, which has the same type as that of the collection:

```
user=> (def a (empty '[1 2 3 4]))
#'user/a
user=> a
[]
```

   * = is used to check equality between one collection and another completely different collection:

```
user=> (= '[1 2 3] '[1 2 3])
true
user=> (= '[1 2 3] '(1 2 3))
true
user=>(= '[1 2 3]  1)
false
```